

Solver Specification:

highOrderElectroActivationFoamImplicitJfNK

A high-order, implicit, Jacobian-free Newton–Krylov monodomain solver in OpenFOAM with an Eigen linear-algebra back-end

Pablo

May 14, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver **highOrderElectroActivationFoamImplicitJfNK**. The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Radial-Extrapolation (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at LRE cell quadrature points; time integration follows a θ -scheme (backward Euler or Crank–Nicolson); the resulting nonlinear coupled system can be solved by Picard fixed-point iteration, a diagonal I_{ion} linearisation, or a Jacobian-Free Newton–Krylov (JFNK) method with Armijo backtracking line search, linear extrapolation initial guess and a physical-bracket clamp on the input to the ionic ODE; the JFNK inner linear system is solved by an in-house restarted GMRES with modified Gram–Schmidt orthogonalisation built on top of the Eigen sparse linear algebra library; the per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; activation times are tracked by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. All algorithmic parameters are exposed through the `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties` dictionaries, and the companion Python driver `run_convergence.py` generates these dictionaries from a single configuration file. This solver is algorithmically identical to its PETSc-based sibling **highOrderElectroActivationFoamImplicitJfNK_PETSc**; the only difference is the linear-algebra library (Eigen vs. PETSc).

Contents

1	Notation	3
2	Governing equations	3
2.1	Monodomain PDE	3
2.2	Ionic ODE	3
2.3	Manufactured MMS equation used for convergence transfer	4
2.4	External stimulus	4
3	Time discretisation: the θ-scheme	4
4	Spatial discretisation	5
4.1	Standard orthogonal FV Laplacian	5
4.2	High-order LRE diffusion	5
4.3	Mass matrices	5
4.4	High-order reconstruction at ionic quadrature points	5
4.5	High-order cell averaging of I_{ion}	6

5	Nonlinear solver	6
5.1	Shared residuals and stopping tests	6
5.2	Picard fixed-point method	6
5.3	Diagonal I_{ion} linearisation	7
5.4	Jacobian-Free Newton–Krylov	7
5.5	In-house restarted GMRES	7
5.6	Armijo backtracking line search	8
5.7	Initial guess, ODE clamp and rollback	8
6	Linear solver: Eigen back-end	8
7	Ionic ODE integration	9
8	TNNP numerical protection	9
9	Activation tracking and benchmarks	9
10	Solver flow chart	10
11	Manufactured driver: run_convergence.py	11
11.1	Geometry and parameter sweep	11
11.2	Nonlinear method and convergence controls	12
11.3	JFNK and diagonal-Ion controls	13
11.4	Linear PDE solver	14
11.5	State ODE integrator	14
11.6	LRE high-order parameters per p-level	14
11.7	Diagnostics and run-control	14
A	Dimension table	15
B	File map	15
C	Differences vs. the PETSc sibling	15

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V
$\mathbf{M}, \mathbf{K}, \mathbf{L}$	Mass, stiffness, scaled diffusion matrix	—
$\mathbf{A}, \mathbf{B}$	Implicit LHS / RHS operators of the θ -scheme	—
$\mathbf{F}(\mathbf{x})$	Discrete nonlinear residual to be zeroed	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]. In physical current-density notation one may write

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi C_m I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk. The factor χC_m multiplying I_{ion} in (1) reflects the convention used by the ionic-model interface: the solver stores `Iion` as a voltage rate with dimensions V s^{-1} , not as an A m^{-3} volumetric current.

Dividing (1) by χC_m gives the form actually used inside the solver,

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -I_{\text{ion}} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

This is exactly what `rhsFromIion` assembles in the solver.

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current returned to the PDE is the algebraic `Iion.cm` of TNNP [1].

2.3 Manufactured MMS equation used for convergence transfer

The convergence driver in `tutorials_electro/highOrderManufacturedFDAImplicitJfNK` solves a manufactured problem with the same numerical structure as the electro-activation solver: implicit monodomain-like PDE, local state ODEs, nonlinear ionic source, optional LRE reconstruction for V_m and I_{ion} , and selectable `Picard/diagonalIon/JfNK` nonlinear coupling. The analytical fields are

$$V_m^*(\mathbf{x}, t) = \sqrt{1+t} F(\mathbf{x}), \quad (4)$$

$$u_1^*(\mathbf{x}, t) = (1+t) G(\mathbf{x}) + V_m^*(\mathbf{x}, t), \quad (5)$$

$$u_2^*(\mathbf{x}, t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \quad (6)$$

$$u_3^*(\mathbf{x}, t) = 0, \quad (7)$$

with, for example, $F = \cos(\pi x) \cos(2\pi y)$ and $G = 1 + x y^2$ in 2D. In 3D, $F = \cos(\pi x) \cos(2\pi y) \cos(3\pi z)$ and $G = 1 + x y^2 z^3$. The state equations are

$$\dot{u}_1 = (u_1 + u_3 - V_m)^2 u_2^2 + \frac{1}{2}(u_1 + u_3 - V_m) u_2^2 (V_m - u_3), \quad (8)$$

$$\dot{u}_2 = -(u_1 + u_3 - V_m) u_2^3, \quad (9)$$

$$\dot{u}_3 = 0. \quad (10)$$

The manufactured ionic source is

$$I_{\text{ion}}^{\text{MMS}}(V_m, \mathbf{u}) = -\frac{1}{2}(u_1 + u_3 - V_m) u_2^2 (V_m - u_3) + \frac{\beta}{\chi C_m} (V_m - u_3), \quad (11)$$

and the PDE source used by the manufactured solver is $-I_{\text{ion}}^{\text{MMS}}$. The coefficient β is the spatial eigenvalue of the trigonometric manufactured field under the constant conductivity tensor:

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d = 1, \\ -\pi^2 (D_{xx} + 4D_{yy}), & d = 2, \\ -\pi^2 (D_{xx} + 4D_{yy} + 9D_{zz}), & d = 3. \end{cases}$$

Thus the MMS case is not intended to reproduce TNNP physiology; it is a controlled problem with a known exact solution that exercises the same discrete operators and nonlinear solution paths used by this solver.

2.4 External stimulus

The external current is a time- and region-windowed step:

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \quad b = 1, \dots, B, \\ 0 & \text{otherwise,} \end{cases} \quad (12)$$

with $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$ as Niederer-benchmark defaults.

3 Time discretisation: the θ -scheme

Time is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by

$$\frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \theta \mathbf{L} \mathbf{V}^{n+1} + (1 - \theta) \mathbf{L} \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \quad (13)$$

implicitScheme	θ	Scheme
backwardEuler	1	First-order, L-stable
crankNicolson	1/2	Second-order, A-stable

After multiplication by Δt^{-1} the implementation forms

$$[\mathbf{M}/\Delta t - \theta \mathbf{L}] \mathbf{V}^{n+1} = [\mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}] \mathbf{V}^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n, \quad (14)$$

with the two Eigen `SparseMatrix<scalar, RowMajor>` operators

$$\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}, \quad \mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}. \quad (15)$$

$\mathbf{A}$ and $\mathbf{B}$ are rebuilt at every time step by copying $\mathbf{M}$, scaling by Δt^{-1} and adding $\pm \theta \mathbf{L}$. When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and $\mathbf{B} = \mathbf{M}/\Delta t$. The scaled diffusion is $\mathbf{L} = \mathbf{K}/(\chi C_m)$.

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` computes the classical 5/7-point Laplacian. Per internal face f with face area $\mathbf{S}_f$,

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad (16)$$

contributing $-a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{own}]$, $+a_f/V_{\text{own}}$ to $\mathbf{K}[\text{own}, \text{nei}]$, and the transposed pair to the neighbour row. `zeroGradient/empty` patches contribute zero flux. `fixedValue` patches contribute to the owner diagonal only.

4.2 High-order LRE diffusion

When `useHighOrder_Vm = true` and $d > 1$, `assembleHighOrderStiffnessMatrix` integrates the flux on a face quadrature with reconstructed gradients:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right), \quad (17)$$

where $\mathcal{S}_f$ is the LRE face stencil (~ 15 – 50 cells) and $\mathbf{g}_{f,q,j} = \mathbf{QRGradFaceGPCoeffs}$. The contribution of each $\mathbf{V}_j$ is added to row `own` with weight $+\Phi_{f,j}/V_{\text{own}}$ and to row `nei` with weight $-\Phi_{f,j}/V_{\text{nei}}$, yielding a negative semi-definite Laplacian.

4.3 Mass matrices

Two options:

Lumped (`massMatrix lumped`). `assembleDiagonalMassMatrix` writes $\mathbf{M} = \mathbf{I}$.

Consistent HO (`massMatrix consistent`). `assembleConsistentMassMatrixHO` fills

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad \phi_j(\mathbf{x}_q) = \delta_{ij} + \mathbf{g}_{i,j} \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_{i,j} : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_{i,j}^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (18)$$

with $\mathbf{d}_q = \mathbf{x}_q - \mathbf{C}_i$. A safety guard silently downgrades to the lumped form when `useHighOrder_Vm = false` or $d \leq 1$.

4.4 High-order reconstruction at ionic quadrature points

If `useHighOrder_Iion = true`, the ionic ODE is integrated at cell quadrature points. $\mathbf{V}_m$ is reconstructed there by the same Taylor expansion as in (18):

$$V_m(\mathbf{x}_q) \approx V_{m,c} + (\nabla V_m)_c \cdot \mathbf{d}_q + \frac{1}{2} \mathbf{H}_c : (\mathbf{d}_q \otimes \mathbf{d}_q) + \frac{1}{6} \mathbf{T}_c^{(3)} : (\mathbf{d}_q \otimes \mathbf{d}_q \otimes \mathbf{d}_q), \quad (19)$$

implemented in `reconstructVmAtIionIntegrationPoints`.

4.5 High-order cell averaging of I_{ion}

`averageIionIntegrationPointsToCells` contracts the per-QP I_{ion} back to cells:

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q}. \quad (20)$$

5 Nonlinear solver

At each outer time step the unknown is the new voltage vector $\mathbf{x} = \mathbf{V}^{n+1}$. For any candidate $\mathbf{x}$ the solver first reconstructs the corresponding V_m field, then calls `evaluateNonlinearFields`. That routine resets the ionic states to their time-step-start values $\mathbf{s}^n$, integrates the local ODEs from t^n to t^{n+1} using the candidate voltage, obtains $I_{\text{ion}}(\mathbf{x})$, and optionally averages quadrature-point values back to cells. This gives the nonlinear residual

$$\mathbf{F}(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{r}(I_{\text{ion}}(\mathbf{x})), \quad \mathbf{x} = \mathbf{V}^{n+1}, \quad (21)$$

with

$$\mathbf{r}(I_{\text{ion}}) = \mathbf{B}\mathbf{V}^n + \theta \left[-I_{\text{ion}}(\mathbf{x}) + I_{\text{ext}}^{n+1}/(\chi C_m) \right] + (1 - \theta) \left[-I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m) \right]. \quad (22)$$

The three nonlinear methods differ only in how they drive $\mathbf{F}(\mathbf{x})$ toward zero.

5.1 Shared residuals and stopping tests

Every method records the same diagnostics:

$$R_{\text{cpl}} = \frac{\|\mathbf{F}(\mathbf{x}_k)\|_2}{\|\mathbf{x}_k\|_2 + \epsilon}, \quad (23)$$

$$R_{V_m} = \frac{\|\mathbf{V}_k - \mathbf{V}_{k-1}\|_2}{\|\mathbf{V}_{k-1}\|_2 + \epsilon}, \quad (24)$$

$$R_{I_{\text{ion}}} = \frac{\|I_{\text{ion},k} - I_{\text{ion},k-1}\|_2}{\|I_{\text{ion},k-1}\|_2 + \epsilon}, \quad (25)$$

$$R_s = \max_j \frac{\|\mathbf{s}_{j,k} - \mathbf{s}_{j,k-1}\|_2}{\|\mathbf{s}_{j,k-1}\|_2 + \epsilon}. \quad (26)$$

The actual implementation uses `relativeL2Difference` for R_{V_m} and $R_{I_{\text{ion}}}$, and `maxStateRelativeL2Difference` for the state maximum. A nonlinear iteration can stop only after `minNonlinearIterations`; convergence then requires `nonlinearTolerance`, `nonlinearVmTolerance` and `nonlinearIionTolerance`. The state criterion `nonlinearStatesTolerance` is included only when `nonlinearRequireStatesConvergence=true`.

5.2 Picard fixed-point method

The Picard branch treats the source as lagged during the PDE solve. At iteration k the states and ionic source are computed from $\mathbf{x}_k$, then the linear PDE

$$\mathbf{A}\mathbf{V}_* = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) \quad (27)$$

is solved with the Eigen sparse solver selected by `linearSolver`. The next candidate is relaxed,

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \omega(\mathbf{V}_* - \mathbf{x}_k), \quad \omega = \text{nonlinearRelaxation}. \quad (28)$$

After the update the solver re-evaluates states and I_{ion} at $\mathbf{x}_{k+1}$ and checks the residuals above. In this electro-activation solver, unlike a pure MMS Picard study, a failed Picard solve rolls back unless `nonlinearAcceptUnconverged=true`.

Advantages. Picard is simple, cheap per nonlinear iteration, and uses the same linear PDE solver as the classical implicit formulation. It is a useful baseline and is often adequate when Δt is small or the ionic source is weakly nonlinear over one time step.

Disadvantages. The source derivative with respect to V_m is ignored in the PDE matrix, so convergence can be slow near the action-potential upstroke or for large Δt . A small residual may require many iterations, and under-relaxation may be needed for robustness.

5.3 Diagonal I_{ion} linearisation

The `diagonalIion` branch is Picard plus a local finite-difference approximation of the source derivative. Define

$$S(V_m) = -I_{\text{ion}}(V_m, \mathbf{s}(V_m)) + I_{\text{ext}}/(\chi C_m).$$

The solver approximates, cell by cell,

$$S'_i(V_{m,k}) \approx -\frac{I_{\text{ion},i}(V_{m,k} + \varepsilon_d) - I_{\text{ion},i}(V_{m,k})}{\varepsilon_d}, \quad \varepsilon_d = \text{diagonalIionEpsilon}. \quad (29)$$

The linearised residual $\mathbf{A}\mathbf{x} - \theta S(\mathbf{x})$ gives the system

$$(\mathbf{A} - \theta \text{diag}(S'_k)) \mathbf{x}_{k+1} = \mathbf{r}(I_{\text{ion}}(\mathbf{x}_k)) - \theta \text{diag}(S'_k) \mathbf{x}_k. \quad (30)$$

The update can still be relaxed with `nonlinearRelaxation`, and the same convergence and rollback logic as Picard is used.

Advantages. It captures the dominant local stiffness of $I_{\text{ion}}(V_m)$ without building a full Jacobian. It often reduces the number of nonlinear iterations relative to Picard while preserving a sparse scalar PDE matrix.

Disadvantages. Only the cell-local diagonal sensitivity is included. Couplings through ODE states, high-order quadrature averaging and spatial reconstruction are not represented in the matrix. The finite-difference step `diagonalIionEpsilon` must be chosen above ODE/noise levels but small enough to be a useful derivative.

5.4 Jacobian-Free Newton–Krylov

JFNK applies Newton to (21):

$$\mathbf{J}(\mathbf{x}_k) \boldsymbol{\delta}_k = -\mathbf{F}(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \boldsymbol{\delta}_k,$$

but never assembles $\mathbf{J}$. GMRES sees only matrix-vector products approximated by

$$\mathbf{J}(\mathbf{x}_k) \mathbf{v} \approx \frac{\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}(\mathbf{x}_k)}{\varepsilon}, \quad \varepsilon = \varepsilon_0 \frac{1 + \|\mathbf{x}_k\|_2}{\|\mathbf{v}\|_2}, \quad (31)$$

with $\varepsilon_0 = \text{jfnkEpsilon}$. Every evaluation of $\mathbf{F}$ re-runs the ODEs and recomputes I_{ion} from the same time-step-start state snapshot. This is expensive but gives a genuinely coupled nonlinear residual.

5.5 In-house restarted GMRES

`solveGMRES` implements GMRES(m) with modified Gram–Schmidt. It solves the Krylov least-squares problem for the Newton correction using at most `jfnkMaxKrylovIterations` basis vectors per restart and at most `jfnkMaxRestarts` restarts. The stopping test is controlled by `jfnkLinearTolerance`. The method is “matrix-free” here because the linear operator is the finite-difference shell in (31), not a stored sparse matrix.

Advantages. JFNK sees the full nonlinear dependence $V_m \mapsto \mathbf{s}(V_m) \mapsto I_{\text{ion}}(V_m, \mathbf{s})$ and usually has far better nonlinear convergence than Picard in stiff regimes. It also avoids explicitly forming a dense or complicated Jacobian.

Disadvantages. Each Krylov vector requires an additional residual evaluation, hence a new ODE integration at every cell or quadrature point. The method is sensitive to the finite-difference perturbation, ODE tolerances and line-search choices. In this Eigen implementation the JFNK inner GMRES has no preconditioner.

5.6 Armijo backtracking line search

The raw Newton update may increase the nonlinear residual when the linear model is poor. If `jfnkLineSearch=true`, the solver tries

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \boldsymbol{\delta}_k, \quad \alpha = \text{nonlinearRelaxation}, \alpha/2, \alpha/4, \dots$$

until the Armijo sufficient-decrease condition holds:

$$\|\mathbf{F}(\mathbf{x}_k + \alpha \boldsymbol{\delta}_k)\|_2^2 \leq (1 - 2c\alpha) \|\mathbf{F}(\mathbf{x}_k)\|_2^2, \quad c = \text{jfnkArmijoC}. \quad (32)$$

Backtracking stops after `jfnkLineSearchMaxIter` attempts or when the step length reaches `jfnkLineSearchAlphaMin`. This globalises Newton: large Newton steps are accepted when they reduce the residual, otherwise they are damped.

5.7 Initial guess, ODE clamp and rollback

The first guess for the time step is either $\mathbf{V}^n$ or the linear extrapolation

$$\mathbf{x}_0 = \mathbf{V}^n + \frac{\Delta t}{\Delta t_{\text{prev}}} (\mathbf{V}^n - \mathbf{V}^{n-1}), \quad (33)$$

selected by `jfnkInitGuessOrder`. It is clamped to `[jfnkInitGuessVmMin, jfnkInitGuessVmMax]`. The same bracket can also be applied to voltage values passed into the ODE solver through `jfnkClampODEInput`; this protects the ionic model during JFNK finite-difference probes.

If the nonlinear loop reaches its iteration budget without satisfying the convergence tests, `nonlinearAcceptUnconverged` decides whether the last iterate is committed. With the default `false`, the solver rolls back V_m , I_{ion} and all ionic states to the beginning of the time step. Rollback prevents a bad nonlinear solve from contaminating the remaining activation simulation.

6 Linear solver: Eigen back-end

The wrapper `solveSparseSystem` drives either of two Eigen sparse solvers on the explicit operator $\mathbf{A}$ (Picard / diagonal paths):

<code>linearSolver</code> key	Eigen type	Type
<code>SparseLU</code>	<code>Eigen::SparseLU</code>	Direct LU
<code>BiCGSTAB</code>	<code>Eigen::BiCGSTAB + IncompleteLUT</code>	Iterative

Both share the `tolerance` and `maxIterations` dictionary keys (the former is ignored by `SparseLU` since it is direct). The matrix-free JFNK branch never instantiates these wrappers; instead it calls the in-house GMRES (Section 5.2) on the FD Jacobian shell.

7 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the OpenFOAM adaptive integrator selected via the `solver` key. Default RKF45 [6]. Keys read from `constant/timeIntegrationProperties`:

Key	Meaning	Default
<code>solver</code>	Concrete ODE integrator	RKF45
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step (ms)	10^{-5}

`relTol` drives the noise floor of the matrix-free Jacobian: $\eta_F \sim \text{relTol}$. Tightening `relTol` buys a lower attainable outer tolerance at the cost of more RKF45 sub-steps per Newton iteration.

8 TNNP numerical protection

The master switch `TNNPNumericalProtection` (default `false`) gates a local guard invoked at every external entry into the embedded TNNP kernel (`calculateCurrent`, `solveODE_start`, `solveODE_end`, `derivatives`).

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMinForRates</code>	scalar (mV)	-200
<code>TNNPVMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6
<code>TNNPProtectionLog</code>	word	<code>TNNP_numericalProtection_summary.dat</code>

When protection is enabled, the guards applied in order are:

1. **Non-finite V** : `!std::isfinite(V) → reset to  $-86.2$  mV.`
2. **V clamp for rates** (if `TNNPCLampVmForRates`): clamp to `[TNNPVMinForRates, TNNPVMaxForRates]`.
3. **$V = 0$ singularity avoidance**: push V off zero by `TNNPVoltageZeroEps` (sign preserving) when $|V|$ falls below that threshold.
4. **Concentration floor** on K_i, Na_i, Ca_i, Ca_{SR} : non-finite or below-floor entries are reset to `TNNPConcentrationFloor`.
5. **Gate clamp** on $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ (if `TNNPCLampGates`): non-finite to zero, clamp to `[0, 1]`.

The embedded implementation stores a capped correction count and emits a compact summary file at exit; it does not depend on external ionic-model logging utilities.

9 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a time step is recorded by linear interpolation:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t, \quad (34)$$

with a per-cell flag cleared after the first crossing so subsequent re-crossings are ignored.

Early termination on P8. When `stopAfterPointActivation = true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored and `effectiveEndTime` is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$ once its $t_{\text{act}} > 0$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd`, with $k = \text{sampleNeighbours}$, $p = \text{samplePower}$.

10 Solver flow chart

The pseudo-code below follows the actual solver. The shared setup is common to all nonlinear methods; then one of the three method-specific blocks is executed according to `nonlinearMethod`.

Algorithm 1 Shared time-step setup and final commit/rollback.

```

1: Read dictionaries and assemble  $\mathbf{M}$ ,  $\mathbf{K}$ ,  $\mathbf{L} = \mathbf{K}/(\chi C_m)$ .
2: while  $t^n < T$  do
3:   Save snapshots:  $\mathbf{V}^n$ ,  $I_{\text{ion}}^n$ ,  $\mathbf{s}^n$ .
4:   Build  $\mathbf{A} = \mathbf{M}/\Delta t - \theta \mathbf{L}$  and  $\mathbf{B} = \mathbf{M}/\Delta t + (1 - \theta) \mathbf{L}$ .
5:   Build old source  $\mathbf{S}^n = -I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)$ .
6:   if jfnkInitGuessOrder  $\geq 1$  and previous step exists then
7:      $\mathbf{x}_0 \leftarrow \mathbf{V}^n + (\Delta t/\Delta t_{\text{prev}})(\mathbf{V}^n - \mathbf{V}^{n-1})$ .
8:     Clamp  $\mathbf{x}_0$  to [jfnkInitGuessVmMin, jfnkInitGuessVmMax].
9:   else
10:     $\mathbf{x}_0 \leftarrow \mathbf{V}^n$ .
11:   end if
12:   Run Algorithm 2, 3 or 4.
13:   if not converged and nonlinearAcceptUnconverged is false then
14:     Roll back  $V_m$ ,  $I_{\text{ion}}$ ,  $\mathbf{s}$  to the saved  $t^n$  snapshots.
15:   else
16:     Commit the final nonlinear iterate.
17:   end if
18:   Update activation times and benchmark samples.
19: end while

```

Algorithm 2 Picard nonlinear solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

```

1: for  $k = 0, \dots, \text{nonlinearIterations} - 1$  do
2:   Store previous iterate  $\mathbf{x}_{k-1}$ ,  $I_{\text{ion},k-1}$ ,  $\mathbf{s}_{k-1}$  for residuals.
3:   EVALUATENONLINEARFIELDS( $\mathbf{x}_k$ ): reset states to  $\mathbf{s}^n$ , reconstruct  $V_m$  at Iion quadrature
   points if enabled, call ionicModel->solveODE, compute  $I_{\text{ion},k}$ .
4:   Build RHS  $\mathbf{r}(I_{\text{ion},k})$  using (22).
5:   Solve  $\mathbf{A} \mathbf{V}_* = \mathbf{r}(I_{\text{ion},k})$  with linearSolver.
6:   Relax:  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \text{nonlinearRelaxation}(\mathbf{V}_* - \mathbf{x}_k)$ .
7:   Re-evaluate ODEs and  $I_{\text{ion}}$  at  $\mathbf{x}_{k+1}$ .
8:   Compute  $R_{\text{cpl}}$ ,  $R_{V_m}$ ,  $R_{I_{\text{ion}}}$  and optional  $R_s$ .
9:   if all requested criteria pass and  $k + 1 \geq \text{minNonlinearIterations}$  then
10:    Mark converged and exit loop.
11:   end if
12: end for

```

Algorithm 3 diagonalIion nonlinear solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

- 1: **for** $k = 0, \dots, \text{nonlinearIterations} - 1$ **do**
 - 2: Evaluate states and $I_{\text{ion},k}$ at $\mathbf{x}_k$ exactly as in Picard.
 - 3: Approximate the local derivative $S'_k \approx -[I_{\text{ion}}(\mathbf{x}_k + \varepsilon_d) - I_{\text{ion}}(\mathbf{x}_k)]/\varepsilon_d$, with $\varepsilon_d = \text{diagonalIionEpsilon}$.
 - 4: Assemble $\mathbf{A}_k = \mathbf{A} - \theta \text{diag}(S'_k)$.
 - 5: Build corrected RHS $\mathbf{r}_k = \mathbf{r}(I_{\text{ion},k}) - \theta \text{diag}(S'_k)\mathbf{x}_k$.
 - 6: Solve $\mathbf{A}_k \mathbf{V}_* = \mathbf{r}_k$ with `linearSolver`.
 - 7: Relax, re-evaluate ODEs/ I_{ion} , compute residuals and test convergence as in Picard.
 - 8: **end for**
-

Algorithm 4 Jacobian-Free Newton–Krylov solve.

Input: Initial candidate $\mathbf{x}_0$, state snapshot $\mathbf{s}^n$.

- 1: **for** $k = 0, \dots, \text{nonlinearIterations} - 1$ **do**
 - 2: Evaluate $\mathbf{F}_k = \mathbf{F}(\mathbf{x}_k)$; this resets states to $\mathbf{s}^n$, solves ODEs and recomputes I_{ion} .
 - 3: Compute residuals. If all requested criteria pass, mark converged and exit.
 - 4: Define matrix-free product $\mathcal{J}_k \mathbf{v} = [\mathbf{F}(\mathbf{x}_k + \varepsilon \mathbf{v}) - \mathbf{F}_k]/\varepsilon$, with ε from `jfnkEpsilon`.
 - 5: Solve $\mathcal{J}_k \boldsymbol{\delta}_k = -\mathbf{F}_k$ by restarted GMRES using `jfnkMaxKrylovIterations`, `jfnkMaxRestarts` and `jfnkLinearTolerance`.
 - 6: **if** `jfnkLineSearch` **then**
 - 7: Start $\alpha = \text{nonlinearRelaxation}$ and backtrack until Armijo (32) succeeds, the trial budget is exhausted, or `jfnkLineSearchAlphaMin` is reached.
 - 8: **else**
 - 9: $\alpha \leftarrow \text{nonlinearRelaxation}$.
 - 10: **end if**
 - 11: $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha \boldsymbol{\delta}_k$.
 - 12: Re-evaluate $\mathbf{F}(\mathbf{x}_{k+1})$, update fields, compute residuals and test convergence.
 - 13: **end for**
-

Legend. `blue` = Python constant set in `run_convergence.py`; `green` = OpenFOAM dictionary key read by the solver. Constants and keys are mapped one-to-one in section 11.

11 Manufactured driver: `run_convergence.py`

The convergence script `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials.electro/highOrderManufacturedFDAImp` generates an MMS case whose numerical path is intended to match this solver. The tables below describe the constants exposed there, the OpenFOAM dictionary entries they generate, and which part of the discrete equations they affect. The same dictionary keys are read by `highOrderElectroActivationFoamImplicitJfNK`; only problem-specific items such as exact fields versus TNNP/stimulus differ.

11.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Affected equation / mechanism
DIMENSIONS	—	Selects 2D or 3D manufactured domain and the corresponding LRE stencils in <code>P_LEVELS_BY_DIM</code> .

Python constant	OpenFOAM key	Affected equation / mechanism
N_VALUES	blockMeshDict cell counts	Spatial refinement. This changes FV/LRE discretisation error in the PDE, Iion quadrature and state reconstruction.
DT_VALUES	controlDict.deltaT	Outer PDE time step. Also sets the integration interval for every local ODE solve.
END_TIME	controlDict.endTime	Final time of the transient MMS test.
IMPLICIT_SCHEMES	spatialIntegrationProperties	Implicit scheme (13): backwardEuler ($\theta = 1$) or crankNicolson ($\theta = 1/2$).
MASS_MATRIX_VALUES	spatialIntegrationProperties	Mass matrix M in $A = M/\Delta t - \theta L$. Chopped or LRE-consistent
ACTIVE_HO_PAIRS_Vm.States	useHighOrder_Vm useHighOrder_Iion LRECoeffs_*	/ Tuple (Vm-level, Iion/states-level). Vm level affects diffusion and consistent mass; Iion/states level affects where ODEs are solved and where the nonlinear source is integrated.
P_LEVELS_BY_DIM	highOrderCoeffs	Maps p1,p2,p3 to LRE polynomial order N, stencil target Nn, radial weight exponent k, and maxStencilSize.

11.2 Nonlinear method and convergence controls

Python constant	OpenFOAM key	Affected equation / mechanism
NONLINEAR_METHOD	nonlinearMethod	Selects the nonlinear strategy for the PDE/source coupling: Picard, JFNK, or diagonalIion.
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Maximum number of nonlinear corrections per PDE time step.
IMPLICIT_MIN_NONLINEAR_ITERATIONS	minNonlinearIterations	Minimum number of corrections before a convergence test may stop the loop.
NONLINEAR_TOLERANCE	nonlinearTolerance	Coupled PDE residual tolerance R_{cpl} . Used directly by JFNK and diagonalIion; in the electro solver it is also reported for Picard.
NONLINEAR_VM_TOLERANCE	nonlinearVmTolerance	Relative correction of the PDE unknown V_m .
NONLINEAR_STATES_TOLERANCE	nonlinearStatesTolerance	Relative correction of all ODE states. In the MMS solver these are u_1, u_2, u_3 ; in this electro solver these are the TNNP state variables.

Python constant	OpenFOAM key	Affected equation / mechanism
NONLINEAR_REQUIRE_STATES_CONVERGENCE	nonlinearRequireStatesConvergence	If true, the state residual is part of the stopping criterion.
NONLINEAR_ACCEPT_UNCONVERGED	nonlinearAcceptUnconverged	If false, a failed electro non-linear solve rolls back V_m , I_{ion} and states to t^n .
NONLINEAR_IION_TOLERANCE	nonlinearIionTolerance	Relative correction of the non-linear source I_{ion} .
NONLINEAR_RELAXATION	nonlinearRelaxation	Picard/diagonal mixing factor or initial Newton step length for JFNK/Armijo.

11.3 JFNK and diagonal-Ion controls

Python constant	OpenFOAM key	Affected equation / mechanism
JFNK_MAX_KRYLOV_ITERATIONS	jfnkMaxKrylovIterations	Restart length m of GMRES(m), controlling memory and Krylov approximation quality.
JFNK_MAX_RESTARTS	jfnkMaxRestarts	Number of times the GMRES basis may be discarded and rebuilt.
JFNK_LINEAR_TOLERANCE	jfnkLinearTolerance	Relative tolerance of the Newton correction solve $J\delta = -F$.
JFNK_EPSILON	jfnkEpsilon	Base perturbation for matrix-free Jacobian-vector products. It should be above the ODE/noise floor but small enough to approximate a derivative.
JFNK_INIT_GUESS_ORDER	jfnkInitGuessOrder	Chooses lagged or linearly extrapolated initial V_m for the nonlinear solve.
JFNK_INIT_GUESS_VM_MIN/MAX	jfnkInitGuessVmMin/Max	Clamp interval for extrapolated guess and, if enabled, ODE inputs.
JFNK_CLAMP_ODE_INPUT	jfnkClampODEInput	Protects the ODE solve from unphysical JFNK finite-difference probes.
JFNK_LINE_SEARCH	jfnkLineSearch	Turns Armijo backtracking on/off.
JFNK_LINE_SEARCH_MAX_ITER	jfnkLineSearchMaxIter	Maximum number of halvings of the Newton step.
JFNK_LINE_SEARCH_ALPHA_MIN	jfnkLineSearchAlphaMin	Floor for the Armijo step length.
JFNK_ARMIJO_C	jfnkArmijoC	Sufficient-decrease parameter in (32).

Python constant	OpenFOAM key	Affected equation / mechanism
DIAGONAL_IION_EPSILON	diagonalIionEpsilon	Finite-difference step for the diagonal source derivative in <code>diagonalIion</code> .

11.4 Linear PDE solver

Python constant	OpenFOAM key	Affected equation / mechanism
IMPLICIT_LINEAR_SOLVER	linearSolver	Eigen sparse solver used by Picard and <code>diagonalIion</code> for $AV = r$. JFNK instead uses GMRES on the Jacobian-free shell.
IMPLICIT_TOLERANCE	tolerance	BiCGSTAB tolerance for the linear PDE solve.
IMPLICIT_MAX_ITERATIONS	maxIterations	BiCGSTAB iteration cap.

11.5 State ODE integrator

Python constant	OpenFOAM key	Affected equation / mechanism
STATE_ODE_SOLVER	<code>stateODESolver</code> <code>timeIntegrationProperties.solveStates</code>	/ Local ODE integrator for MMS states or TNNP states.
STATE_ODE_ABS_TOL	<code>stateODEAbsTol</code> <code>timeIntegrationProperties.absTol</code>	/ Absolute tolerance of the ODE solve.
STATE_ODE_REL_TOL	<code>stateODERelTol</code> <code>timeIntegrationProperties.relTol</code>	/ Relative ODE tolerance. This controls the noise floor seen by JFNK finite differences.
STATE_ODE_INITIAL_STEP	<code>stateODEInitialStep</code> <code>initialODEStep</code>	/ First RKF45 sub-step attempted inside one PDE time step.
STATE_ODE_MAX_STEPS	<code>stateODEMaxSteps</code> / <code>maxSteps</code>	Maximum ODE substeps per local solve.

11.6 LRE high-order parameters per p-level

`P_LEVELS_BY_DIM[dim][p]` gives, for each (dim, p): `N` (polynomial order), `Nn` (stencil neighbours), `k` (search radius), `maxStencilSize` (cap). The script writes these into a nested `highOrderCoeffs{LRECoeffs.Vm{...}; LRECoeffs.Iion{...}}` block whenever HO is enabled for either `Vm` or `Iion`.

11.7 Diagnostics and run-control

Python constant	OpenFOAM key	Affected mechanism
WRITE_NONLINEAR_RESIDUALS	<code>writeNonlinearResiduals</code>	Residual log on/off.
KEEP_LOGS_ON_SUCCESS	—	Python-only switch: keep solver/blockMesh logs after successful runs. Failed-run logs are always kept.

For the electro-activation tutorial the analogous driver also exposes activation-threshold, P8 stopping and diagonal-sampling parameters. They do not affect the PDE/ODE solve itself; they affect only stopping and post-processing of activation times.

A Dimension table

Field	OpenFOAM dimensionSet	SI
V_m	$[1, 2, -3, 0, 0, -1, 0]$	volt
I_{ion}	$\text{dimVoltage}/\text{dimTime}$	V s^{-1}
$\text{externalStimulusCurrent}$	$\text{dimCurrent}/\text{dimVolume}$	A m^{-3}
conductivity	$\text{dimCurrent}^2 \text{dimTime}^3 / (\text{dimMass} \text{dimVolume})$	S m^{-1}
χ	$\text{dimless}/\text{dimLength}$	m^{-1}
C_m	$\text{dimCurrent} \text{dimTime} / (\text{dimVoltage} \text{dimArea})$	F m^{-2}

B File map

Path (relative to cardiacFoam-pc/)	Role
<code>applications/solvers/.../JfNK.C</code>	Main translation unit, including the embedded TNNP CellML kernel, state update, current evaluation, and local ODE integration.
<code>applications/solvers/.../createFields.H</code>	Field initialisation and dictionary reading.
<code>tutorials_electro/.../run_convergence.py</code>	Driver script.
<code>constant/cardiacProperties</code>	χ , C_m , $\mathbf{D}$, ionic model.
<code>constant/timeIntegrationProperties</code>	ODE solver, tolerances, sub-step cap.
<code>constant/spatialIntegrationProperties</code>	Time scheme, HO flags, JFNK / linear / line-search options.
<code>constant/stimulusProtocol</code>	Stimulus regions and timing.

C Differences vs. the PETSc sibling

Algorithmically the two solvers are identical. The only differences are in the linear-algebra back-end and a few user-visible knobs:

Aspect	This solver (Eigen)	PETSc sibling
Sparse matrix container	<code>Eigen::SparseMatrix</code>	PETSc Mat
Sparse direct solver	<code>Eigen::SparseLU</code>	KSPPREONLY+PCLU
Iterative solver (Picard branch)	<code>Eigen::BiCGSTAB+ILUT</code>	KSP+PC (full PETSc catalogue)
JFNK inner Krylov	in-house GMRES	KSPGMRES on MatShell
Preconditioner inside JFNK	none (Krylov-only)	any PETSc PC
Build-time dependency	header-only Eigen3	PETSc + MPI
Parallel run support	serial only	MPI-parallel out of the box

All convergence-related parameters (`nonlinearTolerance`, `jfnkEpsilon`, `jfnkLineSearch*`, `jfnkInitGuess*`, `jfnkClampODEInput`, `nonlinearAcceptUnconverged`, `ODE_*`) have the same names and meanings in both solvers, so the same `run_convergence.py` configuration produces an algorithmically equivalent run with either back-end (modulo the performance and parallel-scaling differences).

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, *AJP – Heart and Circulatory Physiology* **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, *Prog. Biophys. Mol. Biol.* **104** (2012) 22–48.
- [3] D. A. Knoll, D. E. Keyes, *Jacobian-Free Newton-Krylov methods: a survey of approaches and applications*, *J. Comp. Phys.* **193** (2004) 357–397.
- [4] J. Nocedal, S. J. Wright, *Numerical Optimization*, 2nd edition, Springer, 2006.
- [5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd edition, SIAM, 2003.
- [6] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.